

로봇 트윈 MCP 서버 설계서

작성 2026-08-18 · 갱신 2026-08-22 · 구현 완료 (v1)

최초 작성 당시 트윈은 `rdfp` 패키지의 `twin/` 서브패키지였다. 이후 제어 계층 분리로 **독립 패키지** `robot_twin` 이 되었고, 본 문서의 경로는 그 기준으로 갱신되어 있다. 변경 내용 자체는 그대로 이관되었다.

1. 목적과 배경

1.1 무엇을 풀려는가

LLM 에이전트가 **프로그램을 작성하지 않고** 대화만으로 로봇의 상황을 확인하고, 트윈이 제공하는 연산을 조합해 사용자가 요구한 작업을 수행하게 한다.

목표 시나리오는 다음 한 문장이다.

"박스를 반시계 방향 50도로 옮겨줘."

이 요청을 풀려면 에이전트가 (a) scene 에 무엇이 어디 있는지 읽고, (b) 파지 자세를 만들고, (c) 50도 회전한 목표 좌표를 계산하고, (d) 이동·파지·해제를 순서대로 실행해야 한다. **네 가지**가 모두 도구로 제공되어야 조합이 성립한다.

여기에 더해, 이 워크스페이스는 모방학습 데이터 플랫폼이므로 **같은 실행을 학습 데이터로 남기고 싶은 경우가** 있다. 그때만 에피소드 경계를 찍는 도구가 추가로 필요하다 — 조작의 필수 요소가 아니라 **선택적으로 얹는 층**이며, 경계 없이도 위 네 가지는 그대로 동작한다(§5.6·§5.7).

1.2 왜 MCP 서버인가

트윈은 이미 REST API 를 갖고 있으므로 에이전트가 HTTP 를 직접 호출하게 할 수도 있다. **그러나 MCP 를 쓰는 이유는 도구 발견(discovery)과 스키마 검증을 프로토콜이 담당하기 때문이다.** 에이전트는 `tools/list` 한 번으로 무엇을 할 수 있는지 알고, 잘못된 인자는 호출 전에 걸러진다. 엔드포인트 목록을 프롬프트에 적어 넣는 방식은 연산이 늘 때마다 프롬프트를 고쳐야 하고, 그 동기화가 깨지는 순간 조용히 틀린다.

1.3 배치



MCP 서버는 **트윈의 클라이언트일 뿐**이며 ROS 를 알지 못한다. 이 경계를 지킨 것이 설계의 뼈대다 — 자세한 근거는 §5.1.

1.4 기동 — 최소 경로

한 번만 하는 등록과 매번 하는 기동을 구분한다. 섞으면 매번 등록하려 든다.

등록 — 한 번만

MCP 서버는 stdio 로 말하므로 **클라이언트가 프로세스를 띄운다**. 사람이 실행하는 것이 아니라 Claude Code 에 등록해 두는 것이다.

```
cd ~/development/mdtpy/robot-twin
./sbin/register_mcp_server.sh          # 기본 user scope - 모든 프로젝트에서 잡힌다
```

등록 정보는 ~/.claude.json 에 남아 **재부팅해도 유지된다**. mcp_server.py 를 고쳐도 재등록은 필요 없다 — 등록된 명령이 uv run 이라 매번 현재 소스를 쓴다 (세션을 새로 시작하거나 /mcp 에서 재연결하면 반영된다).

다시 실행해야 하는 것은 **등록 내용 자체가 바뀔 때뿐**이다 — 트윈 포트·호스트·ID 변경, 두 번째 트윈 추가 (--name), 프로젝트 디렉터리 이동, 새 PC 세팅.

--scope local 로 등록하면 **등록할 때의 프로젝트 디렉터리에서만** 잡힌다. 로봇 작업은 보통 rdfp_ws 쪽에서 하므로 거기서 도구가 보이지 않는다. 기본이 user 인 이유다.

기동 — 매번

배치도(§1.3)의 **아래에서 위로** 띄운다. 각 층은 아래 층이 없으면 기다리지 않고 실패한다.

```
# 터미널 A - 로봇 스택
rdfp_env # ROS 환경 + overlay + 워크스페이스로 cd
ros2 launch robot_control panda_mock.launch.py

# 터미널 B - 트윈
rdfp_env
./scripts/run_robot_twin.sh

# Claude Code - /mcp 로 확인 (도구 18개면 정상)
```

터미널마다 rdfp_env 가 필요하다. ~/.bashrc 는 ~/.devrc 를 읽어 ros2_env / rdfp_env 를 정의만 한다 — 환경을 켜지는 않는다(옵트인). 무조건 켜면 모든 셀에 PYTHONPATH 가 박혀 ROS 와 무관한 uv 프로젝트까지 오염되기 때문이다. rdfp_env 한 번이 네 가지를 한다 — .ros2rc (배포판 자동 판별 → humble, ROS_DOMAIN_ID=31 , rmw_cyclonedds_cpp) · RDFP_* 변수 · install/setup.bash · 워크스페이스로 cd .

source install/setup.bash 만 하면 조용히 어긋난다. 이 파일이 /opt/ros/humble 을 chain 하므로 ros2 명령은 멀쩡히 돌지만 ROS_DOMAIN_ID 가 비고(=0) RMW 도 기본값(fastrtps)이라, rdfp_env 를 켜면 터미널과 서로를 보지 못한다. "노드는 떴는데 토픽이 안 보인다"로만 나타나 진단이 늦다.

옛 절차인 source ~/.ros2rc 도 심볼릭 링크가 남아 있어 아직 동작하지만, RDFP_* 변수가 빠지고 overlay 를 따로 얹어야 한다. 환경 구성 전체는 [python_env_guide.md §2.3](#) 에 있다.

순서가 중요하다. MCP 서버는 도구 목록을 트윈 카탈로그에서 가져오므로, 트윈보다 먼저 연결하면 tools fetch failed 로 잡힌다. 그 상태면 /mcp 에서 재연결한다.

세션/에피소드 연산(begin_task / end_task)까지 쓰려면 터미널 A 를 ros2 launch rdfp rdfp_panda_mock.launch.py 로 바꾼다 — 사용 설명서 §1.2 참고.

이상할 때

증상	확인
/mcp 목록에 아예 없음	등록 직후라면 Claude Code 세션을 새로 시작
tools fetch failed	터미널 B 의 트윈이 떠 있는지
도구는 보이는데 팔이 안 움직임	터미널 A 의 스택 — 트윈 로그에 MoveGroup client ready 가 찍혔는지

한 번에 좁히려면 ~/development/mdtpy/robot-twin 에서 다음을 실행한다. 서버 기동과 도구 조회를 각각 판정해 어디가 끊겼는지 알려준다.

```
./sbin/run_mcp_server.sh --check
```

전체 사용법은 ~/development/mdtpy/robot-twin/README.md 의 "MCP 서버" 절에 있다.

여기에는 최소 경로만 두고 나머지는 위임한다 — 기동 절차는 경로·extra 이름·

클라이언트 종류에 따라 자주 바뀌어, 두 곳에 두면 어긋난다.

2. 트윈 쪽 변경 — 능력을 클라이언트에게 알리기

MCP 서버가 도구 목록을 만들려면 트윈이 자기 능력을 기계가 읽을 수 있는 형태로 내놓아야 한다. 이 절이 그 요구사항을 만족시키기 위해 기존 구현을 어떻게 고쳤는지 정리한 것이다.

2.1 변경 전 상태와 문제

트윈은 이미 두 카탈로그 엔드포인트를 갖고 있었다.

엔드포인트	내놓던 것
GET /variables	name , source_type , staleness_ms , units , schema_version
GET /operations	name , kind , resource , idempotent , inputs_schema , endpoint

inputs_schema 가 **이미 JSON Schema** 라는 점이 결정적이었다 — MCP 의

Tool.inputSchema 가 요구하는 형식과 같으므로 변환이 필요 없다.

문제는 **설명이 없다는 것**이었다. 설정 파일에는 twin.description 최상위 한 줄만 있었고, 변수와 연산에는 사람이 읽는 주석만 YAML 주석으로 달려 있었다. 주석은 파서가 버리므로 API 로 나갈 수 없다.

이것이 왜 치명적인가 — **LLM 의 도구 선택은 description 에 거의 전적으로 의존한다**. 이름만 있는 도구는 에이전트가 이름에서 의미를 추측하게 만들고, 추측이 틀리면 로봇이 잘못 움직인다.

2.2 변경 내역 (코드·설정 3개 + 문서 1개)

경로는 rdfp_ws/src/robot_twin/ 기준이다. 트윈은 **독립 ROS 패키지**이며 제어 계층에 속한다(package.xml 의 의존은 robot_control / rdfp_msgs 뿐).

(1) robot_twin/config.py — 스키마에 필드 추가

```
class VariableConfig(_Base):
    name: str
    description: str = ''          # 추가
    source: SourceConfig
    ...

class OperationConfig(_Base):
    name: str
    description: str = ''          # 추가
    kind: OperationKind
    ...
```

이 변경이 없으면 YAML 만 고쳐도 트윈이 기동조차 못 한다. 설정 모델의 공통 베이스가

model_config = ConfigDict(extra='forbid') 이기 때문이다 — 선언되지 않은 키는 오타로 간주해 거부한다. 이 정책 자체는 유지하는 것이 맞다(오타를 조용히 넘기지 않는다). 필드를 명시적으로 추가하는 것이 올바른 대응이다.

기본값을 '' 로 둔 이유는 **기존 설정 파일과의 호환**이다. 도커 마운트 등으로 외부 YAML 을 쓰는 배치가 있으므로, 필수 필드로 만들면 그 설정들이 전부 깨진다.

(2) robot_twin/api.py — 카탈로그 응답에 실어 보내기

```
# GET /variables
{'name': v.name, 'description': v.description, 'source_type': ..., ...}

# GET /operations
{'name': o.name, 'description': o.description, 'kind': o.kind, ...}
```

YAML 에만 넣으면 목적을 이루지 못한다. MCP 서버는 HTTP 로만 트윈을 보므로, 설정에 있는 값이 API 로 나오지 않으면 존재하지 않는 것과 같다.

기존 필드 뒤가 아니라 name 바로 뒤에 넣은 것은 응답을 사람이 읽을 때의 순서일 뿐 기능적 의미는 없다.

(3) src/robot_twin/config/robot_twin_panda01.yaml — 설명 19건 작성

변수 8개, 연산 11개 전부에 description 을 채웠다. 작성 원칙은 셋이다.

원칙 1 — "무엇"에서 끝내지 않고 "언제"를 적는다.

```
- name: scene_objects
description: >-
  scene 안 물체들의 현재 위치·자세·크기. ... **물체를 집기 전 파지 좌표를 여기서
  얻는다** - 고정 좌표를 쓰면 물체가 그 자리에 없다. ...
```

에이전트는 도구를 고를 때 "이게 뭔가"보다 "지금 이것 불러야 하나"를 판단한다.

원칙 2 — 완료가 성공을 뜻하지 않는 연산은 그 사실을 적는다.

```
- name: move_linear
description: >-
  ... **완료가 도달을 뜻하지 않는다** - 경로가 잘려도 성공으로 끝날 수 있으므로,
  도달이 중요하면 ee_pose 를 읽어 목표와 비교한다.
```

이걸 적지 않으면 에이전트가 거짓 성공 위에 다음 동작을 실행한다.

원칙 3 — 스키마로 막을 수 없는 함정을 적는다.

joint_states 에는 "move_to_joints 에 그대로 되돌려 주면 finger 관절이 섞여 실패한다"를,
scene_objects 에는 "cylinder 의 dimensions 는 [높이, 반지름]"을 넣었다. 둘 다 타입 검사를 통과하고 런타임에만 드러나는 오류다.

미구현 연산 둘(move_to_pose , move_gripper)에는 **"호출하지 말 것"** 과 대체 연산을 명시했다. MCP 서버가 이들을 감추지만(§4.2), 설명 자체를 방어선으로 한 겹 더 둔 것이다.

(4) 문서 — docs/robot_twin/robot_twin_user_guide.md §8.2

"새 연산 추가" 절의 YAML 예제에 description 을 넣고, **주석이 아니라 데이터**라는 설명을 붙였다. 앞으로 연산을 추가하는 사람이 빠뜨리지 않게 하기 위해서다.

2.3 이 변경이 만든 성질

트윈 설정이 **MCP 도구 정의의 단일 출처**가 되었다. 트윈 YAML 에 연산을 추가하면 MCP 서버 코드를 건드리지 않아도 도구가 늘어난다. 손으로 유지하는 중복 목록이 없으므로 두 곳이 어긋날 여지가 없다.

3. MCP 서버 쪽 변경

프로젝트: ~/development/mdtpy/robot-twin

3.1 기존 자산의 재사용

이 프로젝트는 MCP 이전에 이미 세 층으로 나뉘어 있었고, 그것이 그대로 MCP 도구의 세 층이 되었다.

기존 모듈	역할	MCP 에서의 쓰임
client.py	트윈 HTTP 래퍼. run() 이 sync/async 를 모두 처리	자동 생성 도구의 실행 경로
ops.py	검증 래퍼 (COMPLETED ≠ 도달)	_OVERRIDES 의 구현
pick_n_place.py	기하 헬퍼 + 프레임 보정 (to_arm_command) + 합성 시퀀스	기하 계산 도구 + move_linear 의 프레임 보정 (§6.8)

새로 작성한 로직이 거의 없다는 것이 이 설계의 성과다. MCP 서버는 배선과 정책(무엇을 감추고 무엇을 바꿔 끼울지)만 담당한다.

3.2 변경 내역

파일	상태	내용
src/robot_twin/mcp_server.py	신규	서버 본체 (~505행)
src/robot_twin/client.py	수정	공개 메서드 3개 추가 — read_raw , variables , operations
src/robot_twin/__init__.py	수정	mcp_server 를 import 하지 않는다는 주석
pyproject.toml	수정	mcp optional extra, robot-twin-mcp entry point
README.md	수정	MCP 절 추가
tests/conftest.py	수정	FakeTwin 에 카탈로그 메서드 5개
tests/test_mcp_server.py	신규	단위 37건
tests/test_mcp_protocol.py	신규	프로토콜 왕복 7건

client.py 에 공개 메서드를 추가한 이유

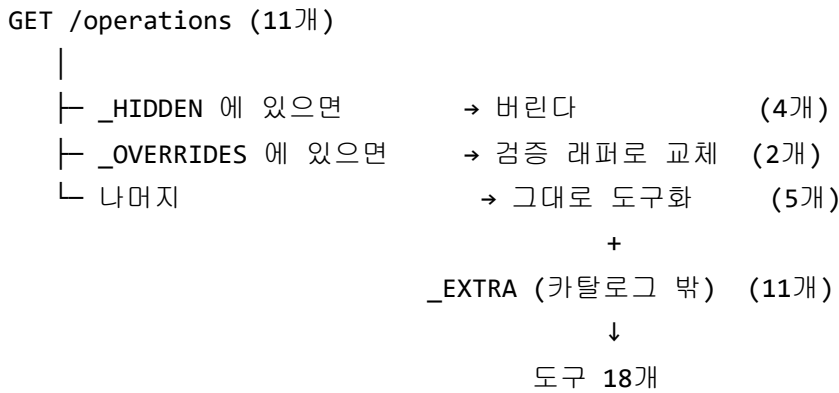
MCP 서버가 처음에는 twin._request('/operations') 처럼 사설 메서드를 직접 찼었다. 두 가지가 문제였다 — (a) 캡슐화를 깨고, (b) 테스트에서 가짜 트윈을 끼우기 어렵다 (사설 메서드까지 흉내 내야 한다). 세 메서드를 공개로 올려 해결했다.

```
def read_raw(self, name: str) -> dict:    # quality/reason 포함 원본
def variables(self) -> list:              # GET /variables
def operations(self) -> list:             # GET /operations
```

read_raw 가 기존 read 와 따로 있는 이유는 **품질을 예외로 바꾸지 않기** 위해서다. read 는 quality != OK 면 예외를 던지는데, 에이전트에게는 "값이 낡았다"는 사실 자체가 판단 근거이므로 그대로 넘겨야 한다.

4. 도구 구성

4.1 생성 규칙



4.2 실제 도구 목록 (18개)

자동 생성 (5) — 카탈로그의 description 과 inputs_schema 를 그대로 쓴다.

```
move_gripper_to_target · move_to_joints · reset_scene · stop_session ·
stop_episode
```

검증 래퍼로 교체 (2)

도구	raw 를 감춘 이유
move_linear	계획의 일부만 실행해도 COMPLETED 로 끝난다. ops.move_linear_verified 가 outputs.final_pose 를 목표와 비교해 reached 를 판정
move_to_named_target	JGPC 스택은 open loop 라 정상 종료해도 도달 보장이 없다. outputs.closed_loop 로 판정하고, 판정 불가면 why 로 사유를 돌려준다

감춤 (4)

도구	이유
move_to_pose , move_gripper	미구현 — 호출하면 EXECUTION_ABORTED
start_session , start_episode	여는 경로를 begin_task 하나로 좁혀 짝을 보장 (§5.6)

추가 (11)

분류	도구
상태	read_state , read_states , list_state_variables , get_resource_status
기하 (로봇을 움직이지 않음)	pose_above , pose_rotated_about_base_z , grasp_pose_of_object
작업 경계	begin_task , end_task
안전	emergency_stop , release_emergency_stop

4.3 설명 보강

카탈로그 설명을 그대로 쓰지 않고 한 가지를 덧붙인다.

```
if op.get('resource'):
    text += f"\n\n점유 자원: {' , '.join(op['resource'])}. 같은 자원을 쓰는 연산이 "
    "실행 중이면 RESOURCE_BUSY 로 거부되므로, 먼저 끝나기를 기다렸다가 다시 부른다."
```

자원 경합은 **트윈 전체의 공통 규약**이라 개별 연산 설명에 적을 성질이 아니다. 그렇다고 빠뜨리면 에이전트가 409 를 받고 같은 호출을 반복한다. 규약은 서버가 붙이고 설정에는 연산 고유의 내용만 둔다.

4.4 목표 시나리오의 조합

"박스를 반시계 50도로 옮겨줘"가 도구로 풀리는 경로다. **조작만 하는 경우**가 기본이다 (§1.1).

<code>read_state('scene_objects')</code>	→ 물체와 그 좌표
<code>grasp_pose_of_object(objects[0])</code>	→ 파지 자세
<code>pose_above(grasp)</code>	→ 접근 지점
<code>move_linear(above) → move_linear(grasp)</code>	→ <code>reached</code> 확인하며 진행
<code>move_gripper_to_target('grasp')</code>	→ <code>stalled</code> 로 파지 판정
<code>pose_rotated_about_base_z(grasp, 50)</code>	→ 목표 좌표 (쿼터니언 포함)
<code>move_linear(...) → move_gripper_to_target('open')</code>	

같은 실행을 학습 데이터로 남기려면 앞뒤에 경계를 두른다. 이때 물체는 `read_state` 가 아니라 `begin_task` 가 돌려준다 — 씬을 새로 배치하면서 그 배치를 서버가 기억해 `end_task` 의 `metadata` 로 넘기기 때문이다 (§5.7).

<code>begin_task(task_label, scene='one_cube')</code>	→ 실제 배치 <code>objects</code> ← <code>read_state</code> 대신
... 위와 동일 ...	
<code>end_task(outcome='success')</code>	→ 배치는 자동으로 <code>metadata</code> 에

5. 개발 중 결정 사항

5.1 MCP 서버를 트윈 안이 아니라 밖에 둔다

결정: 별도 프로세스로 트윈의 HTTP API 를 호출하는 어댑터로 만든다. ROS 노드를 새로 만들지 않는다.

근거: 트윈이 자원 락·ETag·E-stop 의 **단일 중재자**다. MCP 서버가 자체 ROS 노드를 가지면 ROS 클라이언트가 돌이 되고, 그 순간 409 RESOURCE_BUSY 가 무의미해진다 — 락은 트윈을 거치는 요청만 볼 수 있기 때문이다.

부수 효과: MCP 서버가 ROS 를 몰라도 되므로 `source install/setup.bash` 없이 어느 머신에서나 둔다.

5.2 코드 위치 — robot-twin 프로젝트 안

후보: (a) robot-twin 프로젝트 안 (b) 별도 uv 프로젝트 (c) `rdfp_ws` 의 ROS 패키지(`robot_twin`)

결정: (a).

근거: 재사용할 자산(`client.py` / `ops.py` / `pick_n_place.py`)이 전부 거기 있다. (b) 는 path 의존성으로 참조해야 하고 헬퍼를 고칠 때마다 두 저장소를 오간다. (c) 는 ROS 의존이 전혀 없는 코드를 `colcon` 빌드에 묶는다 — 트윈이 별도 패키지가 된 뒤에도 마찬가지다. MCP 서버는 HTTP 만 쓰므로 ROS 워크스페이스에 있을 이유가 없다.

5.3 도구 구성 — 하이브리드

후보: (a) 전부 자동 생성 (b) 전부 손으로 선별 (c) 자동 생성 + 오버라이드

결정: (c).

근거: (a) 는 코드가 가장 적고 트윈과 항상 동기화되지만, `raw move_linear` 가 노출되어 §4.2 의 "거짓 성공" 문제가 그대로 남고 미구현 연산도 새어 나간다. (b) 는 통제가 강하지만 트윈에 연산을 추가할 때마다 MCP 쪽도 고쳐야 해 두 곳이 어긋난다.

(c) 는 **기본은 자동, 위험한 것만 명시적 예외**라는 형태로 양쪽 성질을 모두 얻는다. 예외 목록이 짧게 유지되는 한 유효하다.

5.4 상태 변수를 resource 가 아니라 tool 로 노출

결정: 상태 변수를 MCP resource 가 아니라 **tool** (`read_state` 등)로 노출한다.

근거: MCP 의 세 원시 타입은 *누가 호출을 결정하는가*로 갈린다.

	결정 주체
resource	애플리케이션(호스트) — 사용자가 첨부하거나 앱이 컨텍스트에 넣는다
tool	모델 — 필요할 때 스스로 호출한다

"박스를 옮겨줘"를 받은 에이전트는 **작업 도중에** "지금 물체가 어디 있지?"를 스스로 확인해야 한다. resource 로만 노출하면 클라이언트 지원에 따라 모델이 읽지 못한다.

애초 구상은 "상태 변수 → resource, 연산 → tool" 이었으나 이 이유로 바꾸었다. resource 병행 노출은 v1 범위에서 제외했다(§8).

5.5 설명은 트윈 설정에 두고 API 로 가져온다

후보: (a) MCP 서버에 하드코딩 (b) 트윈 설정에 두고 카탈로그로 받는다

결정: (b).

근거: (a) 를 택하면 연산의 의미가 두 저장소에 나뉘어 살고, 트윈 동작이 바뀌었을 때 설명이 조용히 낡는다. 설명은 연산의 일부이므로 연산이 정의된 곳에 있어야 한다.

대가: 트윈 스키마 변경(§2.2)이 선행 작업으로 필요했다. 그만큼의 값은 한다 — 트윈이 자기 능력을 스스로 설명하는 구조가 되었고, MCP 이외의 클라이언트도 그 설명을 쓸 수 있다.

5.6 여는 연산은 감추고 닫는 연산은 남긴다

결정: `start_session / start_episode` 를 감추고, `stop_session / stop_episode` 는 노출한다. 여는 경로는 `begin_task` 하나뿐이다.

근거: 실패 모드가 비대칭이다.

- 에이전트가 `begin_task` 후 `end_task` 를 잊는다 → **서버가 복구할 수 있다**. 다음 `begin_task` 가 실패로 닫고, 연결이 끊기면 종료 경로가 닫는다.
- 에이전트가 raw `start_episode` 를 직접 부른다 → **서버가 모른다**. 추적하지 않는 에피소드가 열린 채 남고 복구 주체가 없다.

닫는 연산을 남기는 이유는 **이전 실행이 남긴 세션을 에이전트가 스스로 정리할 수 있어야** 하기 때문이다. 여는 쪽만 좁히면 그 목적이 달성되면서 복구 수단은 남는다.

5.7 작업 상태를 서버가 소유하고 metadata 를 자동 병합한다

결정: TwinTools 가 열린 작업의 `scene / seed / objects` 를 들고 있다가 `end_task` 의 metadata 에 자동으로 합친다.

근거: 재현의 근거는 `seed` 가 아니라 **실제 배치**다 — 추출 방식이 바뀌면 같은 `seed` 가 다른 배치를 만든다. 에이전트가 `begin_task` 의 반환값을 기억했다가 `end_task` 에 되돌려 주기를 기대하면, 잊는 순간 재현 불가능한 에피소드가 영구히 남는다(기록이 끝난 뒤에는 고칠 수 없다).

에이전트가 명시한 값이 우선하고 서버 값은 `setdefault` 로 채운다.

5.8 v1 범위 — 합성 스킬 제외

결정: 원시 연산 + 기하 헬퍼 + 작업 경계 래퍼까지. `pick_and_place` 를 도구 하나로 노출하지 않는다.

근거: 합성 스킬은 성공률이 가장 높지만 에이전트가 조합을 배우지 못하고, 변형 요청("중간에 멈춰줘", "순서를 바꿔줘")에 대응할 수 없다. 조합이 실제로 자주 실패하는 것을 확인한 뒤에 추가하는 편이 낫다 — 먼저 넣으면 조합 경로가 잘 도는지 알 수 없다.

5.9 mcp 를 optional extra 로

결정: `pyproject.toml` 의 `[project.optional-dependencies]` 에 둔다.

근거: MCP SDK 가 `pydantic / starlette / uvicorn` 등 30개 이상을 끌고 온다.

RobotTwin 클라이언트만 쓰는 쪽이 그 비용을 질 이유가 없다 — 이 프로젝트는 "표준 라이브러리만 쓴다"가 특징이었다.

따라오는 제약: `__init__.py` 가 `mcp_server` 를 import 하면 안 된다. 그러면 extra 없는 환경에서 `import robot_twin` 자체가 실패한다. 주석으로 명시했고 테스트로 확인한다.

5.10 low-level Server API 사용

결정: 편의 API(`MCPServer`) 대신 `mcp.server.lowlevel.Server` + `add_request_handler` 로 `tools/list` · `tools/call` 을 직접 구현한다.

경위: 설치된 SDK 가 **2.0.0** 이었고 `mcp.server.fastmcp` 가 제거되어 있었다. 후속 편의 API 인 `MCPServer` 는 도구 스키마를 **파이썬 함수 시그니처에서만** 뽑는다 (`add_tool(fn, ...)`). 카탈로그의 `inputs_schema` 를 그대로 쓰려면 시그니처를 동적 합성해야 하는데, 그건 얻는 것보다 잃는 게 많다.

low-level 은 `types.Tool(name=, description=, input_schema=)` 를 직접 만들 수 있어 설계 의도(§5.3)에 정확히 맞는다.

5.11 블로킹 IO 를 스레드로 내린다

결정: 도구 실행을 `asyncio.to_thread(tools.call, ...)` 로 감싼다.

근거: `RobotTwin` 은 `urllib` 기반 동기 호출이고 팔 이동은 수십 초가 걸린다. MCP SDK 는 `asyncio` 이므로 이벤트 루프에서 직접 돌리면 **그동안 상태 조회조차 응답하지 못한다**. 카탈로그 조회(`tools/list`)도 HTTP 이므로 같이 내렸다.

5.12 도구 실패를 예외가 아니라 is_error 결과로

결정: 모든 예외를 잡아 `CallToolResult(is_error=True, content=[...])` 로 돌려준다.

근거: 예외를 그대로 올리면 프로토콜 오류가 되어 **세션이 끊긴다**. 도구 하나가 실패했다고 대화가 끝나면 안 된다. 에이전트는 실패를 읽고 다른 방법을 시도해야 한다.

오류 문장에는 **복구 방법을 함께 담는다** — 오류 메시지가 에이전트의 유일한 피드백 채널이라, 코드만 던지면 같은 호출을 그대로 재시도한다.

코드	덧붙이는 안내
RESOURCE_BUSY	<code>get_resource_status</code> 로 확인하고 끝난 뒤 다시 부른다
PRECONDITION_FAILED	<code>read_state('session_state')</code> 로 상태를 확인한다
INVALID_INPUT	메시지에 허용되는 값이 있으면 그중에서 고른다
NOT_FOUND	<code>list_state_variables</code> 로 확인한다

코드	덧붙이는 안내
TimeoutError	로봇이 아직 움직일 수 있다. <code>read_state('ee_pose')</code> 로 확인

5.13 설명을 한국어로 작성

결정: 트윈 설정의 `description` 과 MCP 도구 설명을 한국어로 쓴다.

근거: 출처인 설계서·사용 설명서가 한국어이고, 최신 모델은 한국어 도구 설명을 문제없이 다룬다.

유보: 이 프로젝트는 로그·예외 메시지를 영어로 쓰는 관례가 있고 `description` 도 기계가 읽는 문자열이라는 점에서 영어가 나올 여지가 있다. 재검토 대상으로 남긴다.

6. 주의사항과 알려진 이슈

6.1 보안 — 인증 없는 경로가 그대로 위임된다

트윈은 **인증·TLS 가 없다**(설정 파일에 명시된 전제 — 노출 범위 통제는 망 수준의 책임). MCP 서버는 그 API 를 LLM 에이전트에게 위임하는 경로이므로, **에이전트가 곧 로봇을 움직일 수 있는 주체**가 된다.

- MCP 서버를 등록한 클라이언트는 트윈과 같은 신뢰 경계 안에 있어야 한다.
- 작업공간 한계·속도 제한을 **프롬프트로 지시하면 안 된다**. 현재 서버는 이를 강제하지 않는다(§8).

6.2 카탈로그를 한 번만 읽는다

`TwinTools._operations()` 는 카탈로그를 캐시한다. 트윈 기동 후 연산 목록이 바뀌지 않는다는 전제인데, **트윈을 재시작하면서 설정을 바꾸면 MCP 서버가 낡은 목록을 유지한다**. 현재는 MCP 서버를 재시작해야 한다.

6.3 서버 상태와 트윈 상태가 어긋날 수 있다

작업 상태(`_task`)는 MCP 서버 프로세스의 메모리에 있다. 서버만 재시작하면 서버는 작업이 없다고 믿지만 트윈에는 세션이 열려 있다.

이를 위해 `begin_task` 가 시작 시 `session_state` 를 읽어 `IDLE` 이 아니면 `stop_session` 을 부르고 경고를 돌려준다. **완전한 해결은 아니다** — 그 세션의 에피소드는 경계를 잃는다.

6.4 동시 호출에 대한 보호가 없다

`_task` 접근에 락이 없다. MCP 클라이언트는 보통 도구를 하나씩 호출하지만 프로토콜은 동시 요청을 허용하므로, `begin_task` 두 개가 겹치면 상태가 꼬일 수 있다. v1 은 단일 세션·순차 호출을 전제한다.

6.5 타임아웃이 두 층에 있다

`RobotTwin.run()` 의 폴링 상한(기본 120초)과 **MCP 클라이언트 쪽 타임아웃**이 별개다. 클라이언트가 먼저 끊으면 로봇은 계속 움직이는데 에이전트는 결과를 못 받는다. 긴 연산이 잦으면 클라이언트 타임아웃을 늘려야 한다.

6.6 mock 백엔드에서는 파지 판정이 서지 않는다

mock 은 물리가 없어 물체가 움직이지 않으므로 `stalled` 가 서지 않고, `scene_objects` 로 작업 성패를 관측할 수도 없다. **mock 은 배선 검증용**이며 실제 파지 확인은 Gazebo 백엔드 연동 이후다.

6.7 SDK 2.0 의 API 안정성

`fastmcp` 제거, `Tool.input_schema` (파이썬 필드명) ↔ `inputSchema` (전송 별칭) 등 1.x 와 차이가 크다. SDK 업그레이드 시 `mcp_server.py` 의 배선부를 다시 확인해야 한다. 프로토콜 왕복 테스트(§7)가 이 회귀를 잡는 장치다.

6.8 좌표 프레임 — 기하 도구는 손끝, 이동 도구가 보정한다

기하 도구(`grasp_pose_of_object` / `pose_above` / `pose_rotated_about_base_z`)는 **손끝(TCP) 기준 pose** 를 돌려준다. 에이전트가 다루는 좌표는 "손끝이 어디 가야 하는가" 여야 이해 가능하기 때문이다.

반면 트윈의 `move_linear` 는 planning group 의 tip link(`panda_link8`)를 움직이고, 손끝은 거기서 약 10.3 cm 더 뻗어 있다. 그래서 `move_linear` **오버라이드가 `to_arm_command()` 로 변환한 뒤 넘긴다.** 보정을 빠뜨리면 5 cm 큐브(중심 `z=0.025`)를 집을 때 손끝이 `z = -0.078` 로 **바닥을 파고든다.**

보정 대상은 이 도구 하나뿐이다 — `move_to_named_target` 은 관절 공간 이동이라 기준 링크가 무관하다. 회귀를 막기 위해 넷을 테스트로 고정했다(§7).

정석은 URDF 에 `panda_hand_tcp` 프레임을 두고 `link_name` 으로 지정하는 것이다. 그러면 이미 녹화된 `/ee_pose` 의 기준이 달라져 데이터셋 호환을 따져야 하므로, 지금은 클라이언트 쪽 보정으로 둔다.

7. 검증

종류	건수	무엇을 지키는가
단위 (test_mcp_server.py)	37	도구 구성 규칙, 검증 래퍼 동작, 기하 도구가 로봇을 건드리지 않음, 작업 경계 짝, 좌표 프레임 보정 , 오류 문장
프로토콜 (test_mcp_protocol.py)	7	초기화 → tools/list → tools/call 왕복, 스키마가 전송을 건디는지, 도구 실패가 세션을 끊지 않는지

프로토콜 테스트를 따로 둔 이유: 단위 테스트는 TwinTools 를 직접 호출하므로 **배선이 틀려도 통과한다**. 메모리 트랜스포트로 진짜 ClientSession 을 붙여 프로토콜 위에서 도는지 확인한다.

특히 고정한 것들:

- move_to_pose / start_episode 가 **프로토콜에도 나타나지 않는다**
- 이름으로 직접 찔러도 감춘 도구는 실행되지 않는다
- begin_task 의 호출 순서가 reset_scene → start_session → start_episode (scene 리셋이 에피소드 밖)
- stop_episode 가 실패해도 stop_session 을 시도한다
- 종료 경로가 열린 작업을 닫는다

8. 남은 일

항목	내용
실 스택 검증	mock 스택에 붙여 "박스를 50도 옮겨줘"를 실제로 시켜 본다. 조합이 자주 실패하면 그때 합성 스킴(\$5.8)을 추가한다
작업공간 한계 강제	move_linear 래퍼에 좌표 범위 검사. 한계값이 셋업에 달려 있어 임의로 넣지 않았다
resource 병행 노출	상태 변수를 MCP resource 로도 노출해 사용자가 컨텍스트에 첨부할 수 있게 한다
카탈로그 갱신	\$6.2 — 명시적 갱신 도구 또는 TTL
다중 트윈	현재 서버 하나가 트윈 하나를 본다. 여러 대는 서버를 여러 개 등록한다

9. 참고

- [robot_twin_user_guide.md](#) — 트윈 REST API, 변수·연산 목록, §8.2 새 연산 추가(`description` 작성 지침)
- [robot_twin_design.md](#) — 트윈 자체의 설계 근거
- [auto_episode_collection_draft.md](#) — 세션/에피소드 경계와 `reset_scene` 의 설계 경위
- `~/development/mdtpy/robot-twin/README.md` — 설치·등록·사용법 (최소 기동 경로는 §1.4)